PROJECT / MPI / STENCIL COMPUTATION

Heat diffusion on a rectangular plate

Scientific question. How does a temperature disturbance decay, and at what subdomain size does communication dominate a nearest-neighbour stencil?

Problem and core scope

Solve the homogeneous heat equation on $\Omega = (0,1)^2$,

$$\partial_t u = \alpha \Delta u, \qquad u|_{\partial\Omega} = 0, \qquad u(x,y,0) = \sin(\pi x)\sin(\pi y),$$

with constant $\alpha > 0$, and $t \in (0, T]$. Produce a sequential solver and an MPI solver with a one-dimensional decomposition into horizontal strips. Support grid sizes not divisible by the number of ranks. Two dimensions and constant diffusivity are sufficient; implicit integration and general geometries are outside the core of this project.

Algorithmic suggestions

Focus on explicit schemes, use $n_x \times n_y$ interior points, $h_x = 1/(n_x + 1)$ and $h_y = 1/(n_y + 1)$, central differences, and, at least at the start, forward Euler:

$$\begin{aligned} u_{i,j}^{m+1} = u_{i,j}^m + \Delta t[&r_x(u_{i-1,j}^m - 2u_{i,j}^m + u_{i+1,j}^m) \\ &+ r_y(u_{i,j-1}^m - 2u_{i,j}^m + u_{i,j+1}^m)], \qquad r_x = \frac{\alpha}{h_x^2}, \quad r_y = \frac{\alpha}{h_y^2}, \end{aligned}$$

where $u_{i,j}^m \simeq u(x_i, y_j, t^m)$.

Choose Δt so that $r_x + r_y \leq \rho\Delta_t/2$, with a safety factor such as $\rho = 0.9$. This should ensure the stability of the scheme. For a specified final time t, shorten the last step so the run ends at exactly that time. Use two contiguous arrays and swap their roles; an in-place update implements a different scheme.

Each rank stores its owned rows and one ghost row on either side. Start with paired `MPI_Sendrecv` exchanges to obtain the old-time-level halo before updating. At physical boundaries, keep the prescribed zero values. Generate initial values locally from global coordinates.

What to implement

1. A grid/indexing abstraction with explicit ownership and boundary handling.
2. A serial timestep function and an MPI halo/timestep loop, sharing the same stencil expression.
3. Error diagnostics, a CSV timing record, and a final field file for plotting.

PROJECT 1 / VALIDATION AND DELIVERY

Heat diffusion: checks and experiments

Validation and acceptance

- Compare against the exact solution $u(x, y, t) = e^{-2\pi^2 \alpha t} \sin(\pi x) \sin(\pi y)$ using a discrete L^2 norm. On three successively refined grids with $\Delta t \propto h^2$, expect approximately second-order total convergence on square grids.
- Do weak and strong scalability tests

Required performance study

Start with 64^2, 128^2, and 256^2 interior grids for convergence. For timing, try 256^2, 512^2, and 1024^2 with a fixed 500-step budget, reducing sizes if needed. Performance cases need not share a final physical time; label them as throughput tests.

Possible work distribution (but it is only a suggestion!)

Student	Main responsibility
A	Discretization, stability bound, and exact-solution tests
B	C++ grid storage, serial stencil, and input validation
C	Strip decomposition, halo exchange, and MPI lifecycle
D	Parallel error reductions and decomposition edge cases
E	Benchmark scripts, field plots, and performance interpretation

Optional extension: choose one

Explicit Euler is simple but not accurate. Furthermore, its stability limit can be hard to compute in more realistic situations. An alternative is to use and adaptive scheme. Here we present a very simple scheme, the RK12 embedded scheme of the family of the embedded Runge-Kutta schemes. Given a tolerance $\tau > 0$, the generic step from time t^m to t^{m+1} with a time step Δt reads

- Compute

$$\begin{cases} k^1_{i,j} = [r_x(u^m_{i-1,j} - 2u^m_{i,j} + u^m_{i+1,j}) + r_y(u^m_{i,j-1} - 2u^m_{i,j} + u^m_{i,j+1})] \\ u^*_{i,j} = u^m_{i,j} + \Delta t k^1_{i.j} \\ k^2_{i,j} = [r_x(u^*_{i-1,j} - 2u^*_{i,j} + u^*_{i+1,j}) + r_y(u^*_{i,j-1} - 2u^*_{i,j} + u^*_{i,j+1})] \\ u^{m+1}_{i,j} = u^m_{i,j} + \frac{\Delta t}{2}(k^1_{i,j} + k^2_{i,j}) \\ \delta = \|\mathbf{u}^{m+1} - \mathbf{u}^*\|_{L^2(\Omega)} \end{cases}$$

- if $\delta \leq tol$ reject step, set $\Delta t \leftarrow \Delta t/2$, and repeat the step;
- if $\delta > tol$ step is accepted and if $\delta > 4.1tol$ you may set $\Delta t \leftarrow 2\Delta t$. Move to the next step.

Accessible starting references

- **Driscoll and Braun: Absolute stability**. Open textbook discussion of the heat equation and the explicit timestep restriction. Translate the one-dimensional reasoning to the two-dimensional stencil above.
- For numerical schemes for the heat equation and discussion on stability you may also refer to any good book on numerical methods.
- **Wikipedia page on Runge-Kutta methods**. Look at the part on embedded schemes.
- **Burkardt: `heat_mpi`**. A small C++/MPI heat-equation example. You may use its communication pattern as an example. But you may also decide to follow your ideas!.

- **LLNL: MPI tutorial**. Read blocking and nonblocking point-to-point operations and collective reductions.